```
[catalog-service,d9e61c8cf853fe7fdf953422c5ff567a,eef9e08caea9e32a]
```

Distributed tracing helps us follow a request through multiple services, so we need another application to test whether it works correctly. Go ahead and make the same changes to Edge Service to support OpenTelemetry. Then run the application as a container from your Docker Compose file:

```
$ docker-compose up -d edge-service
```

Once again, send a few requests to trigger the generation of some logs and traces about your HTTP requests. This time you should go through the gateway:

```
$ http :9000/books
```

Using the trace ID logged by Catalog Service, we can retrieve (*correlate*) all the steps involved in processing the HTTP request to the `/books` endpoint started in Edge Service. Being able to navigate from logs to traces (and the other way around) is extremely useful for getting more visibility into all the steps involved in processing a request throughout a distributed system. Let's see how it works in the Grafana stack.

Open a browser window, go to Grafana (http://localhost:3000), and log in with the credentials configured in Docker Compose (`user/password`). On the Explore page, check the logs for Catalog Service (`{container_name="/catalog-service"}`), much like we did earlier. Next, click on the most recent log message to get more details. You'll see a Tempo button next to the trace identifier associated with that log message. If you click that, Grafana redirects you to the related trace using data from Tempo, all in the same view (figure 13.9).